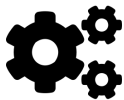


GitHub Actions en détail

Lecture commentée de ci.yml et ci-cd.yml



Lint & Tests



Build



Push GHCR

Objectif : comprendre chaque ligne des deux workflows pour pouvoir les adapter.

Anatomie d'un workflow GitHub Actions

Workflow (un fichier .yml)

Job « lint »

runs-on: ubuntu-latest

1. checkout
2. setup-python
3. pip install ruff
4. ruff check

Job « tests »

runs-on: ubuntu-latest

1. checkout
2. setup-python
3. pip install -r req.txt
4. pytest --cov

Trois niveaux à retenir : Workflow (le fichier) → Jobs (qui tournent en parallèle par défaut) → Steps (séquentiels dans un job).

Quand le workflow se déclenche-t-il ?

La section on: décrit les évènements qui lancent le workflow. On peut en combiner plusieurs.

```
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
```



Astuce : on peut aussi utiliser `schedule (cron)`, `workflow_dispatch (manuel)`, `workflow_call (réutilisation)`.

Ce que ça veut dire :

- Push sur main → workflow lancé
- PR ouverte / mise à jour vers main → workflow lancé
- Push sur autre branche (sans PR) → ignoré
- Création d'un tag → ignoré ici (cf. ci-cd.yml)

Le job lint expliqué ligne par ligne

```
lint:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-python@v5
      with:
        python-version: "3.10"
        cache: pip
    - run: pip install ruff
    - run: ruff check anidata_scraper/ tests/
```

runs-on

VM fournie par GitHub. ubuntu-latest = gratuit, démarre en ~10s.

checkout

Récupère le code du commit qui a déclenché le workflow.

setup-python

Installe Python. cache: pip accélère les runs suivants (×3-5).

run

Exécute une commande shell sur le runner. C'est notre lint.

Tester sur plusieurs versions en parallèle

La matrice multiplie le job pour chaque combinaison. Ici 1 axe (python-version) × 2 valeurs = 2 runs en parallèle.

```
tests:
  strategy:
    fail-fast: false
    matrix:
      python-version: ["3.10", "3.11"]

  steps:
    - uses: actions/setup-python@v5
      with: { python-version: ${ matrix.python-
version }} }
```

Résultat à l'exécution :



Tests · Python 3.10



Tests · Python 3.11

fail-fast: false = on continue même si un run échoue, pour voir si le bug est lié à une version précise.

Trois optimisations à connaître



concurrency

Annule les runs obsolètes : un nouveau push sur la même branche annule l'ancien. Économise du temps de runner.

```
group: ci-${{ github.ref }}  
cancel-in-progress: true
```



cache pip

Réutilise les dépendances déjà téléchargées. Premier run : ~40s. Runs suivants : ~10s.

```
with:  
  cache: pip
```



artifacts

Permet de télécharger des fichiers produits par le run (rapport de couverture, logs, build).

```
uses: actions/upload-artifact@v4  
with:  
  name: coverage-report  
  path: coverage.xml
```

De ci.yml à ci-cd.yml : qu'est-ce qui change ?

ci.yml (Séance 2)

Boucle CI seule

- Job lint
- Job tests (matrice)
- Pas de build d'image
- Pas de push
- Permissions minimales

ci-cd.yml (Séance 3)

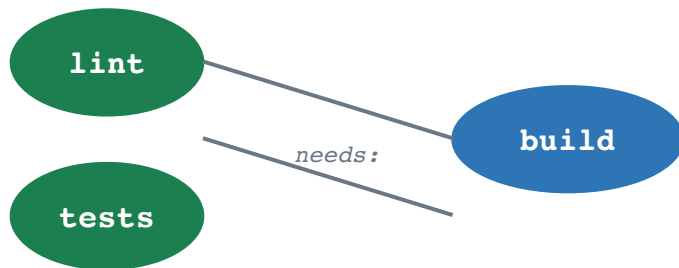
CI + déploiement (CD)

- Job lint
- Job tests (matrice)
- Job build & push (nouveau)
- Permissions: packages: write
- Tagging sémantique automatique

Permissions et dépendances entre jobs

Le job de build ne démarre que si lint ET tests passent. Sécurité et économie de runner.

```
permissions:  
  contents: read  
  packages: write  
  
jobs:  
  build-and-push:  
    needs: [lint, tests]  
    runs-on: ubuntu-latest
```



packages: write est requis pour pousser sur GHCR. Sans cette permission, le job échoue silencieusement à l'étape de login.

Build de l'image et tagging automatique

```
- name: Login GHCR
  uses: docker/login-action@v3
  with:
    registry: ghcr.io
    username: ${ github.actor }
    password: ${ secrets.GITHUB_TOKEN }

- name: Génération des tags
  uses: docker/metadata-action@v5

- name: Build & Push
  uses: docker/build-push-action@v5
```

Tags générés automatiquement :

:latest	<i>push sur main</i>
:sha-a1b2c3d	<i>toujours (SHA du commit)</i>
:1.2.3	<i>tag v1.2.3 créé</i>
:1.2	<i>tag v1.2.x (raccourci)</i>

Le SHA du commit dans le tag permet de revenir à une image précise en cas de problème en production.

À retenir



Un workflow = un fichier YAML qui décrit des jobs (parallèles) faits de steps (séquentiels).



ci.yml valide le code (lint + tests). Toujours déclenché sur push et PR.



ci-cd.yml ajoute un job build-and-push qui ne démarre que si la CI est verte (needs:).



Le tagging automatique (latest, sha-xxx, version sémantique) donne traçabilité et rollback.



Permissions packages: write requise pour pousser sur GHCR. À configurer aussi côté repo GitHub.

Place au TP : un commit sur une PR fait tourner la CI. Un merge fait tourner la CI/CD complète.